



HCMUS

Viet Nam National University
Ho Chi Minh City
University of Science

fit@hcmus

Seminar: Design Patterns

Topic: Chain of Responsibility

Course: CS202 - Programming Systems

Supervisor:

PhD. DINH Ba Tien

MSc. TRUONG Phuoc Loc

MSc. HO Tuan Thanh

Group 03

Full Name	Student ID
Đỗ Viết Hoàng Long	25125024
Trần Như Khải	25125045
Trần Đăng Khoa	25125056
Trần Minh Khoa	25125057

Contents

1	Introduction	2
1.1	Overview & Context	2
1.2	Motivation	2
1.3	Objectives and Scope	2
1.4	Report Organization	2
2	Chain of Responsibility Pattern	3
2.1	Problem Statement	3
2.1.1	Scenario Description	3
2.1.2	Traditional Approach and Naïve Design	3
2.1.3	Architectural Limitations	3
2.1.4	Design Goal	4
2.2	Proposed Solution	4
2.2.1	Core Concept and Architectural Idea	4
2.2.2	Architectural Components	4
2.2.3	Refactored Implementation	5
2.2.4	Resolution of Design Limitations	6
2.3	Evaluation	7
2.3.1	Key Strengths	7
2.3.2	Limitations and Trade-offs	7
2.3.3	Comparative Analysis with Related Design Patterns	7
2.4	Applications	8
2.4.1	Middleware Pipelines in Web Frameworks	8
2.4.2	Event Propagation in GUI Systems	8
2.4.3	Hierarchical Logging Frameworks	8
2.5	Quiz	8
3	References	11

1 Introduction

1.1 Overview & Context

In modern software engineering, the ability to design systems that are flexible, maintainable, and scalable is of paramount importance. Design patterns serve as standardized, reusable solutions to common problems in Object-Oriented Design (OOD), enabling developers to build robust and adaptable architectures. Among the various categories of design patterns, behavioral patterns specifically address the interaction and responsibility distribution among objects, thereby improving communication flow within a system.

1.2 Motivation

A recurring challenge in application development is the need to process a request through multiple sequential steps, such as authentication, authorization, validation, and logging. Traditional implementations based on monolithic `if-else` or `switch-case` constructs often result in tightly coupled code, reduced flexibility, and poor maintainability. Such approaches also violate key design principles, particularly the Open/Closed Principle (OCP) [1]. The **Chain of Responsibility (CoR)** pattern [2] provides a more elegant solution by decoupling the sender of a request from its potential receivers, allowing multiple objects to handle the request dynamically.

1.3 Objectives and Scope

This report aims to deliver a comprehensive and critical analysis of the Chain of Responsibility pattern within the context of the CS202 course. The main objectives are as follows:

- To formulate and analyze the underlying problem that motivates the use of the CoR pattern.
- To examine the architectural structure and design principles associated with CoR.
- To evaluate its implementation trade-offs, including advantages and limitations.
- To illustrate real-world applications and provide practical evaluation scenarios.

Furthermore, this report emphasizes not only theoretical understanding but also practical applicability in real-world system design.

1.4 Report Organization

The remainder of this report is organized as follows:

- **Section 2:** Presents a detailed analysis of the Chain of Responsibility pattern, including problem formulation, architectural design, evaluation, real-world applications, and assessment questions.
- **Section 3:** Lists the references and supporting literature used in this report.

2 Chain of Responsibility Pattern

2.1 Problem Statement

2.1.1 Scenario Description

Consider an enterprise-level application that processes incoming user requests, such as an Online Banking or E-Commerce system. Before executing any core business operation, each request must traverse multiple sequential processing stages to ensure security, correctness, and system stability. These stages typically include:

1. **Authentication:** Verifying whether the user is properly authenticated.
2. **Authorization:** Ensuring that the user has sufficient permissions to access the requested resource.
3. **Data Validation:** Sanitizing and validating input data to prevent invalid input and security vulnerabilities (e.g., injection attacks).
4. **Logging & Rate Limiting:** Recording request metadata for monitoring purposes and preventing abuse through request throttling.

Such a processing pipeline is fundamental in real-world systems, where each step represents a distinct responsibility within the overall request lifecycle.

2.1.2 Traditional Approach and Naïve Design

A straightforward implementation of this pipeline is to encapsulate all processing steps within a single monolithic class using nested conditional logic (e.g., `if-else` statements):

```
1 struct Request {
2     bool isAuthenticated;
3     bool isAuthorized;
4     bool isValid;
5 };
6
7 class RequestHandler {
8 public:
9     void process(const Request& request) {
10         if (authenticate(request)) {
11             if (authorize(request)) {
12                 if (validate(request)) {
13                     // Execute actual core logic
14                     std::cout << "Executing business logic...\n";
15                 }
16             }
17         }
18     }
19 private:
20     bool authenticate(const Request& r) { return r.isAuthenticated; }
21     bool authorize(const Request& r) { return r.isAuthorized; }
22     bool validate(const Request& r) { return r.isValid; }
23 };
24
```

Listing 1: Naïve request processing using nested conditionals

While this approach may appear simple and effective for small-scale systems, it quickly becomes problematic as the number of processing steps increases.

2.1.3 Architectural Limitations

As the system evolves, the monolithic design introduces several critical architectural limitations:

- **Violation of the Open/Closed Principle (OCP) [1]:** Introducing new processing steps (e.g., data decryption or auditing) requires direct modification of the `RequestHandler` class, increasing the risk of unintended side effects.

- **Tight Coupling:** The request sender is tightly coupled with concrete processing logic, reducing modularity and limiting reuse of individual components.
- **High Complexity and Low Maintainability:** Deeply nested conditional structures significantly increase cyclomatic complexity, making the system harder to understand, test, and debug.
- **Rigid Execution Flow:** The processing sequence is hardcoded, making it difficult to dynamically modify, extend, or reorder processing steps at runtime.

These limitations highlight the need for a more flexible and modular architectural approach.

2.1.4 Design Goal

To address these challenges, the system requires a design mechanism that allows multiple independent handlers to process a request in a sequential and decoupled manner. Such a mechanism should support dynamic composition, extensibility, and maintainability, while eliminating direct dependencies between the request sender and specific processing components.

This naturally leads to the adoption of the Chain of Responsibility pattern as a structured solution to these architectural challenges.

2.2 Proposed Solution

To overcome the limitations of high coupling, rigid control flow, and poor maintainability inherent in the naïve conditional approach, we propose restructuring the system using the **Chain of Responsibility (CoR)** behavioral design pattern [2]. This pattern introduces a more modular and extensible mechanism for processing requests through a sequence of independent components.

2.2.1 Core Concept and Architectural Idea

The fundamental principle of the Chain of Responsibility pattern is to decouple the sender of a request from its potential receivers. Rather than embedding all validation logic within a single monolithic structure, the processing workflow is decomposed into a series of independent objects known as **handlers**, each responsible for a specific task.

These handlers are organized into a sequential chain, forming a processing pipeline. When a request enters the system:

1. The request is first received by the initial handler in the chain.
2. Each handler evaluates the request based on its own responsibility and determines whether to process, transform, or reject it.
3. If the request remains valid, it is forwarded to the next handler via a reference, continuing the chain execution.

This delegation-based mechanism eliminates the need for explicit control flow logic in the client and promotes a cleaner separation of concerns.

2.2.2 Architectural Components

The proposed architecture consists of four primary components:

- **Handler (Interface / Abstract Class):** Defines a common interface for handling requests and for linking to the next handler in the chain.
- **BaseHandler (Optional):** Provides a default implementation for chaining behavior by maintaining a reference to the next handler and forwarding the request when appropriate.
- **Concrete Handlers:** Specialized subclasses that encapsulate distinct processing logic (e.g., `AuthenticationHandler`, `AuthorizationHandler`, `ValidationHandler`), each focusing on a single responsibility.
- **Client:** Responsible for constructing and configuring the handler chain dynamically, and for initiating the request processing by invoking the first handler.

This modular decomposition significantly improves extensibility and reusability across different system contexts.

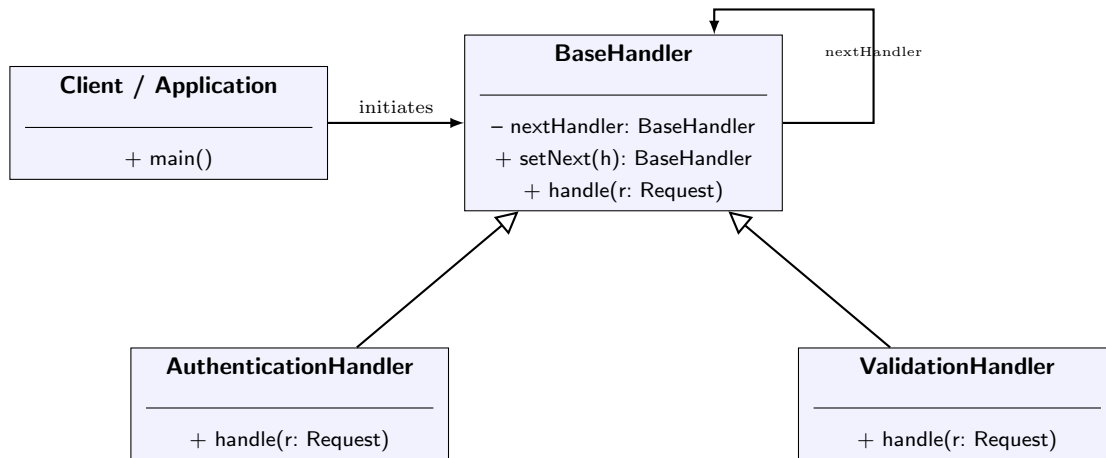


Figure 2.1: UML Class Diagram for Chain of Responsibility Pattern (Authors' illustration)

2.2.3 Refactored Implementation

The following C++ implementation demonstrates how the Chain of Responsibility pattern replaces monolithic conditional logic with a flexible, object-oriented structure:

```

1 #include <iostream>
2 #include <memory>
3
4 // Request definition
5 struct Request {
6     bool isAuthenticated;
7     bool isValid;
8 };
9
10 // 1. Base Handler Abstract Class
11 class BaseHandler {
12 private:
13     std::shared_ptr<BaseHandler> nextHandler;
14
15 public:
16     virtual ~BaseHandler() = default;
17
18     std::shared_ptr<BaseHandler> setNext(std::shared_ptr<BaseHandler> next) {
19         this->nextHandler = next;
20         return next; // Enables fluent chaining
21     }
22
23     virtual void handle(const Request& request) {
24         if (nextHandler) {
25             nextHandler->handle(request);
26         }
27     }
28 };
29
30 // 2. Concrete Handlers
31 class AuthenticationHandler : public BaseHandler {
32 public:
33     void handle(const Request& request) override {
34         if (!request.isAuthenticated) {
35             std::cout << "[FAIL] Authentication failed. Terminating chain.\n";
36             return;
37         }
38         std::cout << "[SUCCESS] Authentication passed.\n";
39         BaseHandler::handle(request);
40     }
41 };
42
43 class ValidationHandler : public BaseHandler {

```

```

44 public:
45     void handle(const Request& request) override {
46         if (!request.isValid) {
47             std::cout << "[FAIL] Data validation failed. Terminating chain.\n";
48             return;
49         }
50         std::cout << "[SUCCESS] Data validation passed.\n";
51         BaseHandler::handle(request);
52     }
53 };
54
55 class CoreLogicHandler : public BaseHandler {
56 public:
57     void handle(const Request& request) override {
58         // Execute actual core logic
59         std::cout << "Executing business logic...\n";
60         BaseHandler::handle(request);
61     }
62 };
63
64 // 3. Client Configuration and Execution
65 int main() {
66     auto authHandler = std::make_shared<AuthenticationHandler>();
67     auto validHandler = std::make_shared<ValidationHandler>();
68     auto coreHandler = std::make_shared<CoreLogicHandler>();
69
70     // Build the chain
71     authHandler->setNext(validHandler)->setNext(coreHandler);
72
73     Request userRequest = {true, true};
74     authHandler->handle(userRequest);
75
76     return 0;
77 }

```

Listing 2: Chain of Responsibility Pattern Implementation

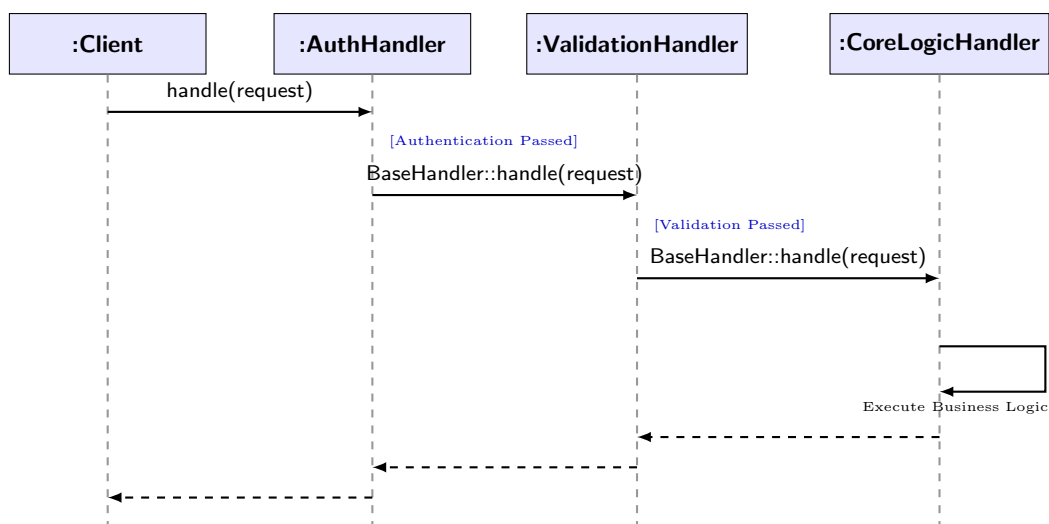


Figure 2.2: UML Sequence Diagram showing successful request execution flow (Authors' illustration)

2.2.4 Resolution of Design Limitations

By adopting the Chain of Responsibility architecture, the system effectively resolves the structural shortcomings of the traditional approach:

- **Single Responsibility Principle (SRP) [1]:** Each handler encapsulates a single processing concern, leading to clearer separation of logic and improved maintainability.
- **Open/Closed Principle (OCP) [1]:** New processing steps can be introduced by extending the handler hierarchy without modifying existing, stable components.

- **Loose Coupling:** The client interacts only with the head of the chain, remaining independent of the internal composition and implementation details of subsequent handlers.
- **Runtime Flexibility:** The processing chain can be dynamically assembled, reordered, or extended at runtime, enabling adaptive system behavior based on context or configuration.

Overall, this design transforms a rigid and tightly coupled structure into a flexible, scalable, and maintainable processing pipeline.

The following section further evaluates the effectiveness of this pattern through practical analysis and real-world applications.

2.3 Evaluation

While the Chain of Responsibility (CoR) pattern introduces a high degree of structural flexibility and modularity, a rigorous architectural evaluation must consider both its strengths and inherent trade-offs in practical system design.

2.3.1 Key Strengths

- **Strong Adherence to OCP and SRP [1]:** Each processing responsibility is encapsulated within an independent concrete handler, ensuring clear separation of concerns. New functionality can be introduced by extending the handler hierarchy without modifying existing, stable components.
- **Dynamic and Flexible Execution Flow:** The sequence of handlers can be configured and reconfigured at runtime, enabling context-aware processing pipelines (e.g., activating an `IpFilterHandler` under high-security conditions).
- **Loose Coupling and Improved Modularity:** The request sender is decoupled from the specific processing logic, interacting only with the entry point of the chain. This abstraction enhances system modularity and promotes component reuse.

2.3.2 Limitations and Trade-offs

Despite its architectural advantages, the CoR pattern introduces several non-trivial trade-offs that must be carefully evaluated:

1. **Uncertain Request Handling Guarantee:** Since each handler independently decides whether to process or forward the request, there is no built-in guarantee that a request will be handled. Improper chain configuration may result in requests reaching the end of the chain without being processed.
2. **Increased Debugging Complexity:** The dynamic and distributed nature of the processing chain makes it more challenging to trace execution flow during debugging. The control flow is no longer centralized, requiring developers to analyze interactions across multiple handler layers.
3. **Performance and Latency Overhead [3]:** Sequential delegation through multiple handlers introduces additional method calls and stack operations. In performance-critical or high-throughput systems, this overhead may become significant if the chain is excessively long, especially due to dynamic dispatch in polymorphic calls.

2.3.3 Comparative Analysis with Related Design Patterns

To better justify the adoption of CoR in practical scenarios, it is essential to compare it with other related design patterns. Table 2.1 summarizes the key distinctions:

Table 2.1: Comparative Analysis of CoR vs. Related Patterns

Pattern	Primary Intent	Key Difference from CoR
Chain of Responsibility	Passes a request through a sequence of handlers.	Any handler may process, modify, or terminate the request flow.
Decorator	Dynamically extends object behavior.	Focuses on behavior composition without altering execution flow.
Strategy	Encapsulates interchangeable algorithms.	Selects a single algorithm at runtime rather than executing multiple steps sequentially.
Command	Encapsulates requests as objects.	Emphasizes request abstraction and execution control, not chained processing.

This comparison highlights that CoR is particularly suitable for scenarios requiring sequential, conditional processing, where multiple components may participate in handling a request.

Overall, the effectiveness of the Chain of Responsibility pattern depends heavily on careful chain design and appropriate use within the system context.

2.4 Applications

The Chain of Responsibility (CoR) pattern is extensively utilized in modern software systems, particularly in scenarios that require flexible and decoupled request processing across multiple components.

2.4.1 Middleware Pipelines in Web Frameworks

A prominent application of CoR can be observed in middleware architectures of contemporary web frameworks. Technologies such as Express.js (Node.js) [4], ASP.NET Core [5], and Spring Security [6] implement request-processing pipelines where each middleware component performs a specific task, such as CORS validation, authentication, or logging. Each layer processes the incoming HTTP request and explicitly delegates control to the next component via a `next()` function, thereby forming a configurable and extensible processing chain.

2.4.2 Event Propagation in GUI Systems

CoR is also fundamental to event handling mechanisms in graphical user interface (GUI) frameworks. In systems such as HTML DOM event propagation [7], Java Swing, and macOS AppKit, user-generated events (e.g., mouse clicks or keyboard inputs) are propagated through a hierarchy of UI elements. If a component does not handle an event, it forwards the event to its parent container, continuing the propagation process until a suitable handler is found or the event reaches the root of the hierarchy.

2.4.3 Hierarchical Logging Frameworks

Logging infrastructures commonly employ a variant of CoR to manage log message handling across multiple output channels. Frameworks such as Apache Log4j [8] route log events to multiple appenders based on the logger hierarchy and additivity settings. Additionally, each appender or logger can have its own independent filters to accept, reject, or conditionally forward log events. This is more complex than a simple severity threshold per appender; the event traverses the logger hierarchy and its attached appenders and filters, forming a sophisticated, multi-layered processing chain.

2.5 Quiz

To evaluate understanding of the Chain of Responsibility pattern, consider the following technical scenarios:

Question 1: Architectural Intent

Scenario: You are designing a system where an incoming request must be processed by *all* validation steps in sequence, regardless of whether a previous step succeeded or failed. Is standard Chain of Responsibility the optimal choice?

(A) Yes, CoR is strictly designed to execute all handlers unconditionally.

- (B) No, because standard CoR allows any handler to break the chain execution prematurely upon failure or completion.
- (C) Yes, but only if implemented with multiple threads.
- (D) No, Strategy pattern must be used instead.

Answer: (B). Standard CoR gives each handler the ability to stop propagation. While many real-world architectures (like the authentication-authorization-validation example) use a processing/filter chain variant of CoR where the chain halts upon validation failure but continues on success, if a scenario requires that *all* steps unconditionally execute regardless of intermediate failures or successes, a pure pipeline or sequential processing model is more suitable.

Question 2: Runtime Behavior

Scenario: Consider a chain composed of three handlers: `HandlerA` $\rightarrow$ `HandlerB` $\rightarrow$ `HandlerC`. If `HandlerB` processes a request and does not call the next handler, what is the outcome?

- (A) `HandlerC` will still execute automatically.
- (B) A runtime exception is thrown.
- (C) `HandlerC` is never invoked, and processing stops at `HandlerB`.
- (D) The request is reprocessed from `HandlerA`.

Answer: (C). In CoR, continuation depends explicitly on invoking the next handler. If omitted, the chain terminates at that point.

Question 3: Advanced Design Consideration

Scenario: You are implementing a logging system where multiple handlers must process the same request independently (e.g., writing to a file, sending alerts, and storing metrics), and no handler should block others from executing. Which design adjustment is most appropriate?

- (A) Use standard CoR without modification.
- (B) Modify CoR so that each handler always forwards the request after processing.
- (C) Replace CoR with an Observer (Publish-Subscribe) pattern.
- (D) Use recursion to re-trigger the chain for each handler.

Answer: (C). When multiple independent consumers must react to the same event without affecting each other, the Observer pattern is a better fit. CoR is inherently sequential and control-flow driven, whereas Observer enables parallel, decoupled notifications.

Question 4: Design Principles and Coupling

Scenario: Which of the following design principles is most directly supported when a developer adds a new security check to an existing Chain of Responsibility pipeline without modifying the existing handlers or the client?

- (A) Liskov Substitution Principle (LSP)
- (B) Open/Closed Principle (OCP)
- (C) Interface Segregation Principle (ISP)
- (D) Dependency Inversion Principle (DIP)

Answer: (B). The Chain of Responsibility pattern allows developers to introduce new handlers to the chain (extension) without altering the existing client or handler code (modification), which perfectly aligns with the Open/Closed Principle.

Question 5: Performance Implications

Scenario: In a highly performance-critical, real-time trading system, a very long Chain of Responsibility is used to validate incoming transactions. Which of the following is a potential drawback of this approach?

- (A) High memory consumption due to large handler object sizes.
- (B) Deadlocks due to circular references in the chain.
- (C) Increased latency overhead due to deep call stacks and dynamic dispatch.
- (D) Inability to process requests asynchronously.

Answer: (C). Sequential delegation through multiple handlers introduces additional method calls, stack operations, and potential dynamic dispatch overhead. In systems with strict latency requirements, an excessively long chain can become a performance bottleneck.

3 References

- [1] Robert C Martin. Design principles and design patterns. *Object Mentor*, 1(34):597, 2000.
- [2] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.
- [3] Scott Meyers. *Effective C++: 55 Specific Ways to Improve Your Programs and Designs*. Pearson Education, 2005.
- [4] Express.js. Using express middleware. <https://expressjs.com/en/guide/using-middleware.html>, 2024. Accessed: Aug 2026.
- [5] Microsoft. Asp.net core middleware. <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/>, 2024. Accessed: Aug 2026.
- [6] Spring. Spring security architecture. <https://docs.spring.io/spring-security/reference/servlet/architecture.html>, 2024. Accessed: Aug 2026.
- [7] MDN Web Docs. Event bubbling and capture. https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Building_blocks/Events#event_bubbling_and_capture, 2024. Accessed: Aug 2026.
- [8] Apache Software Foundation. Log4j 2 architecture. <https://logging.apache.org/log4j/2.x/manual/architecture.html>, 2024. Accessed: Aug 2026.